

Document Summary

This note provides an overview of artificial intelligence (AI) and machine learning (ML), exploring definitions, approaches like the Turing Test and rational agents, and key subfields such as natural language processing and computer vision. It delves into ML paradigms including supervised, unsupervised, and reinforcement learning, emphasizing data handling, model selection, and addressing biases. The content highlights the shift from rule-based to data-driven programming, equipping readers with foundational knowledge for AI and ML applications.

Introduction to AI and Machine Learning

What is AI?

This section explores various definitions and approaches to **artificial intelligence (AI)**. It focuses on human-like thinking, rational decision-making, and key subfields. By examining these perspectives, we can understand AI as a field that aims to create systems capable of performing tasks that typically require human intelligence.

Making Computers Think Like a Human

To make computers think like humans, we first need to understand human cognition. This involves studying how the human brain processes information and makes decisions.

- How does a human think?
 - Brain imaging techniques from neuroscience reveal neural activity and brain structures involved in thinking.
 - Psychological experiments from cognitive science test mental processes like memory, perception, and problem-solving.
- Alison Gopnik, a developmental scientist, has contributed insights into how children learn, which informs models of human-like AI learning.

This approach emphasizes mimicking biological processes, but it faces challenges in fully replicating the complexity of the human brain.

Human-Like AI

AI systems designed to imitate human cognitive processes, drawing from neuroscience and cognitive science.

The Turing Test Approach

The **Turing Test** provides a practical way to evaluate AI by assessing whether a machine can imitate human intelligence convincingly.

Turing Test

This test evaluates a machine's ability to exhibit intelligent behavior equivalent to, or indistinguishable from, a human's. Proposed by Alan Turing in 1950, it involves a human judge conversing with both a human and a machine via text, trying to identify which is which. If the machine fools the judge, it passes.

To succeed in the Turing Test, AI systems must demonstrate several core capabilities. These form the building blocks of intelligent behavior:

- **natural language processing** - Enables communication through understanding and generating human language.
- **knowledge representation** - Allows storing and organizing knowledge in a usable form.
- **automated reasoning** - Supports answering questions and drawing logical conclusions from stored knowledge.
- **machine learning** - Helps detect and extract patterns from data, adapting to new circumstances over time.
- **computer vision** - Facilitates perceiving and interpreting visual objects in the environment.
- **robotics** - Enables physical manipulation of objects in the real world.

An analogy to aeronautical engineering illustrates this: The goal of the field is making machines that fly so exactly like pigeons that they can fool other pigeons. In other words, the focus is on behavioral imitation rather than internal mechanisms.

However, the Turing Test has notable issues that limit its reliability as a measure of true intelligence:

- Variability in protocols and judges can lead to inconsistent results.
- Success often depends on deception rather than genuine understanding!
- Early chatbots, like Eliza from 1966, performed well using “cheap tricks” such as pattern matching simple responses, without real comprehension.

To address these flaws, alternative benchmarks like the **Winograd Schema** have been developed. These are multiple-choice questions that humans can easily answer using common sense, but computers struggle with if they rely on superficial tricks.

Winograd Schema Example

The sack of potatoes had been placed below the bag of flour, so it had to be moved first. What had to be moved first?

- *The sack of potatoes*
- *The bag of flour*

(The correct answer requires understanding the spatial relationship: the potatoes block the flour.)

Resource: <https://ptchallenge-workshop.github.io/>

This test highlights the need for deeper reasoning in AI.

Idealized or “Right” Way of Thinking

Another perspective on AI emphasizes formal, logical methods to achieve correct thinking, independent of quirks.

Logic in AI

This refers to patterns of argument that always yield correct conclusions from correct premises. For instance, if all humans are mortal (premise) and Socrates is human (premise), then Socrates is mortal (conclusion).

Logician Approach to AI

In this method, problems are described in formal logical notation (like propositional or first-order), and general deduction procedures are applied to solve them automatically.

Despite its appeal, this approach encounters significant challenges:

- Describing real-world problems and knowledge in precise logical notation is often impractical due to the complexity.

- Handling uncertainty—such as incomplete or probabilistic information—is difficult within strict logical frameworks.
- The computational complexity of finding solutions from a large list of facts can make it infeasible for real-time applications.

These limitations have led to hybrid approaches that incorporate flexibility.

AI Computational Rationality

A more modern view frames AI around rational decision-making under constraints, prioritizing over mimicking human processes.

Rational Agent

This is an entity that achieves the best possible outcome, or under uncertainty, the best expected outcome. Rationality is defined by actions that align with goals, not by internal thought processes.

- *Goals expressed in terms of outcome utility, a quantitative measure of desirability (e.g., higher utility for winning a).*
- *It accommodates uncertainty by considering probabilities.*
- *Rationality focuses on decisions made, without requiring-like cognition.*
- *Ultimately, rationality means maximizing expected utility: the sum of each possible outcome's utility weighted by its probability.*

Utility optimization account for the agent's computational constraints, leading to concepts like bounded rationality (optimal within limits) or bounded optimality (best achievable given resources).

In this framework, AI equates to **Computational Rationality**—designing systems that compute rational actions efficiently.

The skills from the Turing Test framework rational action in practical ways:

- **knowledge representation** - Stores knowledge to inform decisions.
- **automated reasoning** - Answers questions and draws conclusions to enable good decisions.
- **natural language processing** - Facilitates communication to generate comprehensible sentences and navigate complex social environments.
- **machine learning** - Detect patterns and adapts to new circumstances, improving effective behavior over time.

- **computer vision** - Perceives objects to gather environmental data.
- **robotics** - Manipulates objects to interact with the physical world beyond language.

Resources for further exploration:

- <https://developer.nvidia.com/deep-learning> (Deep learning tools and tutorials)
- <http://www-inst.eecs.berkeley.edu/~cs188/fa19/> (Berkeley AI course)
- <http://slazebni.cs.illinois.edu/fall17> (Illinois AI materials)

#computer-science/artificial-intelligence

Machine Learning

Machine learning (ML) is a core subfield of AI emphasizes algorithms learning patterns from data to improve performance on specific tasks without being explicitly programmed for every scenario. This builds on AI foundations by diving into how machines "learn" from experience. See [Artificial Intelligence](#) for broader context.

Definition of Learning

To formal learning in machines, we use a structured definition that ties experience to improved performance.

Machine Learning Definition

- 1. *Experience E , which typically consists of data or interactions.*
- 2. *Task T , the specific problem the system aims to solve.*
- 3. *Performance measure P , a metric to evaluate success on the task.*

An agent learns if its performance at T , as measured by P , improves with experience E .

The ultimate goal is to enable computers to perform tasks well without requiring manual programming of all competencies—letting the system generalize from examples.

Here are concrete examples to illustrate this definition:

Task T	Experience E	Performance Measure P
Playing checkers	Playing practice games against itself	Percentage of games won against an arbitrary opponent

Task T	Experience E	Performance Measure P
Recognizing hand-written words	Database of human-labeled images of handwritten words	Percentage of words classified
Driving on four-lane highways using vision sensors	A sequence of images and steering commands recorded while observing a human driver	Average distance traveled before a human-judged error
Categorize email messages as spam or legitimate	Database of emails, some with human-given labels	Percentage of email messages correctly classified

Learning Examples

These examples show how learning applies across diverse domains, from games to real-world applications.

Resource: <http://www-inst.eecs.berkeley.edu/~cs188/fa19/>

Programming with Data

Traditional programming often struggles with complex, real-world problems, leading to the rise of data-driven approaches in ML.

We seek adaptive, robust, and fault-tolerant systems that can handle variability without breaking.

Rule-based implementations, while common, have drawbacks:

- They are difficult for programmers to create comprehensively.
- They are brittle, missing many edge cases in dynamic environments.
- Maintaining them explicitly becomes a nightmare as rules proliferate.
- Overall, they prove ineffective for scalable tasks.

In the traditional paradigm: Computer + Data + Program → Output. The program encodes all logic explicitly.

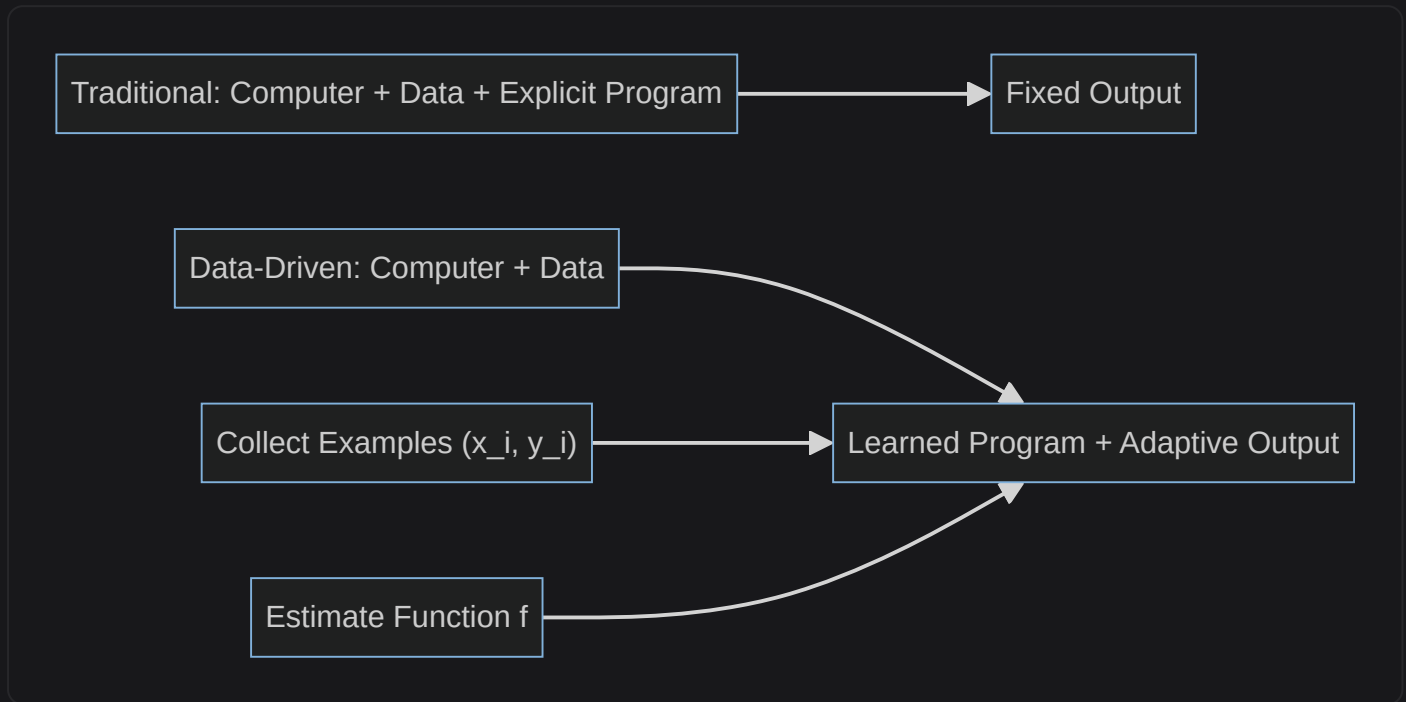
An alternative shifts the burden: It's often easier to obtain examples of desired outputs than to write perfect rules.

- Instead of hard-coding "IF x THEN DO y," collect many input-output pairs (x_i, y_i) .
- Estimate a function f such that $f(x_i) \approx y_i$ —this is supervised learning.
- For unlabeled data, detect underlying patterns—this is unsupervised learning.

This represents a fundamental shift: Computer + Data → Output + Program (where the program emerges from learning from data).

Figure credits: G. Hinton, A. Smola

To visualize this paradigm shift, consider the following Mermaid diagram showing the transition from rule-based to data-driven programming:



Supervised Learning

Supervised Learning

This involves learning from labeled data, where inputs are paired with correct outputs. The model trains to predict labels for new inputs.

- **Classification:**
 - Binary Classification: Outputs are binary, e.g., $y = \{-1, 1\}$ for negative/positive classes (like spam/not spam).
 - Multiclass Classification: Outputs span multiple categories, e.g., $y = \{1, \dots, k\}$ for k classes (like digit recognition: 0-9).
- **Regression:**
 - Linear: Predicts continuous values with a straight-line relationship, e.g., house price based on size.
 - Non-linear: Handles curved relationships, e.g., stock price prediction incorporating multiple factors.

Supervised Learning Example

In binary classification for email spam detection, labeled emails (spam or not) train a model to classify new ones.

Slide credits: A. Ng, A. Smola, E. Eaton

Unsupervised Learning

Unsupervised Learning

Unsupervised Learning involves working with unlabeled data to discover hidden patterns or structures, such as clustering similar items or reducing dimensionality.

Unsupervised Learning Example

For instance, grouping customers by shopping habits without predefined categories reveals market segments.

Figures credit: E. Eaton

Many More Types of Learning

Machine learning encompasses a variety of paradigms beyond the basics, each suited to different challenges.

Reinforcement Learning

Agents learn by interacting with an environment, receiving rewards or penalties to maximize long-term gains. It's like training a dog with treats—trial and error guides optimal actions.

Reinforcement Learning Example

A robot learning to navigate a maze by receiving positive rewards for reaching the goal and negative for hitting walls.

Figures credit: Sutton & Barto

Other types include:

- Active Learning: The model queries for labels on uncertain examples to improve efficiency.
- Online Learning: Updates incrementally as new data arrives, ideal for streaming scenarios.
- Multi-task Learning: Trains on multiple related tasks simultaneously to share knowledge.
- Metric Learning: Learns distance measures between data points for better similarity judgments.
- Ensemble Learning: Combines multiple models (e.g., random forests) for robust predictions.
- Self-Supervised Learning: Generates labels from the data itself, like predicting masked words in text.
- Discriminative vs. Generative Learning: Discriminative models focus on boundaries between classes; generative models learn the full data distribution.
- [...]

These expand ML's applicability to dynamic, interactive settings. See [Reinforcement Learning](#) for deeper exploration.

#computer-science/machine-learning

Who is a Machine Learner?

A machine learner is someone who designs and implements systems that learn from data—bridging domain expertise with technical skills.

- XKCD comic: <https://xkcd.com/1838/> (Humorously depicts the iterative, often frustrating nature of ML workflows.)

Key skills for a machine learner:

- Framing a Learning Problem: Identifying the task, data, and metrics clearly.
- Designing a Learning System:
 - Choose the training experience (e.g., what data to use).
 - Choose exactly what is to be learned (the target function, like a classifier).
 - Choose a learning algorithm to infer the target function from the experience (e.g., neural networks or decision trees).

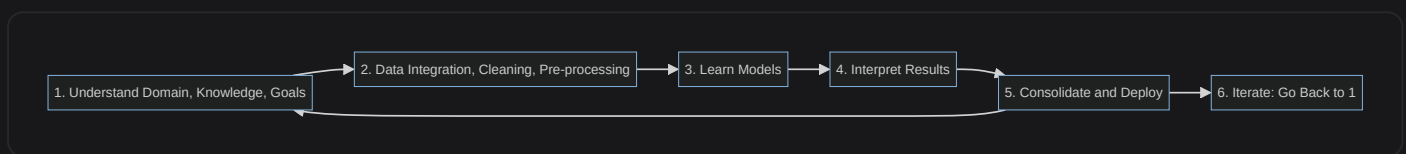
Pseudo-Algorithm for Machine Learning

This outlines a cyclical process for building ML systems:

1. Understand the domain, prior knowledge, and goals to set the context.
2. Data integration, selection, cleaning, pre-processing, etc., to prepare quality inputs.
3. Learn models by applying algorithms to the data.
4. Interpret results to validate and gain insights.
5. Consolidate and deploy discovered knowledge into production.
6. Go back to 1 for iteration and refinement.

Based on a slide of Ray Mooney.

The following Mermaid diagram illustrates this pseudo-algorithm as a cyclical flowchart for clarity:



Dealing with Data

Data is the fuel of AI and ML, but its volume and variety pose challenges. In just one minute online in 2025, an immense amount of activity generates data:

- 694M Spotify songs streamed
- 231M emails sent
- 6.3M Google searches conducted
- 3.47M YouTube videos watched
- 625M TikTok videos watched
- \$43.6M spent online during peak shopping times
- 174K apps downloaded
- 66K Instagram photos shared
- 2.1M active Facebook users

This explosion underscores the need for effective data handling. See [Data Handling in ML](#) for advanced techniques.

How to Deal with Data

Handling data systematically ensures reliable models. The key steps are:

1. Observe the data: Examine raw samples to understand its nature and quality.

2. Summarize their uncertainty': Use statistics like mean, variance, or distributions to quantify variability.
3. Search for constraints and invariances (prior knowledge): Identify rules or patterns that hold across examples, incorporating domain expertise.
4. Find a model to explain the data: Select or build a representation that captures underlying structures.
5. Represent the data: Transform it into features suitable for learning (e.g., numerical vectors).
6. Find a solution to learn both the representation and the model for the data by exploiting advanced computing capabilities: Leverage GPUs or distributed systems for complex training.
7. Avoid data bias: Actively check for and mitigate skewed representations that could lead to unfair outcomes.

Datasaurus Dataset

This example demonstrates how summary statistics can mislead. All these datasets have the same summary stats to 2 decimal places (mean $x=54.26$, mean $y=47.83$, sd $x=16.76$, sd $y=26.93$, correlation= -0.06), yet they look completely different visually—highlighting the importance of exploration beyond aggregates.

Resource: <https://www.autodeskresearch.com/publications/samestats>

Probabilistic reasoning a crucial role here, using probabilities to model uncertainty (e.g., Bayesian methods for updating beliefs with new data). See [Probability in AI](#).

Models for Describing the Data

At the heart of ML is building models that map inputs to outputs based on observed data.

Data Modeling

Main idea: Observe the world, identify knowledge about it, and use that to construct a model as a map from input (x) to output (y). For example, a model might predict $y = f(x)$, where f is learned from data pairs.

Slide credit: E. Rodolà

Choosing a Model Family

The same data can be described in different ways, raising the question: What is the "" way? Overly simple models underfit (miss patterns), while overly complex ones overfit (memorize noise).

Multiple visualizations showing different model fits on the same data illustrate this trade-off—e.g., a line vs. a wiggly curve on scatterplot points.

Choosing the right family (e.g., linear regression vs. neural networks) balances expressiveness with generalization.

Slide credit: E. Rodolà

Choosing a Representation

Representation is key: How we encode data as features affects model performance.

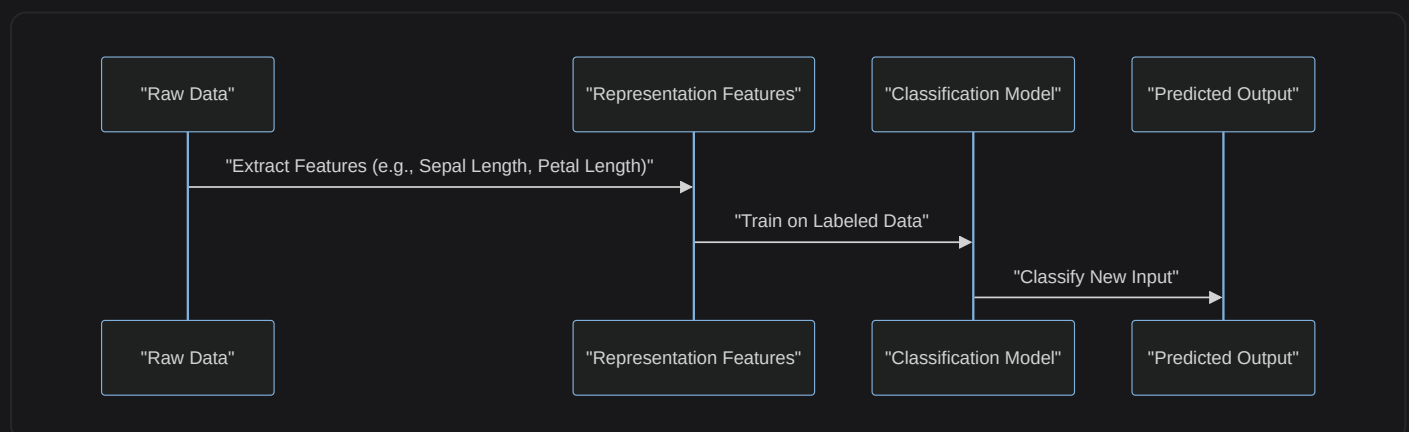
Iris Flower Representation

In iris flower classification, using Sepal Length and Petal as representation features.

A classification model then processes these features. Techniques like linear and non-linear dimensionality reduction (e.g., PCA for linear, t-SNE for non-linear) simplify-dimensional data while preserving structure.

Slide credits: Various for figures.

To clarify the flow from raw data to classification, here's a Mermaid sequence diagram:



Deep Learning

Deep learning extends ML by using multi-layered neural networks to automatically learn hierarchical representations from data. See [Neural Networks](#) for prerequisites.

Deep Learning

A task-driven paradigm to extract patterns and latent features from observations. Layers progressively abstract from raw inputs (e.g., pixels to edges to objects).

Features are instrumental for the task and drive decisions—they aren't fixed but emerge during training.

Features are Task-driven

Features are meaningful only in the context of a task! For example, in card games, color might matter for ranking suits but not for poker hands, where rank and suit combinations define value. Generic features like color become task-relevant based on the problem.

Deep Learning in Visual Classification

In visual classification (e.g. identifying flowers), raw images are transformed via features like sepal/petal measurements into a classification model outputting species.

Visualizations: Sepal Length, Petal Length → Representation Features → Classification Model → Output.

Slide credit: E. Rodolà

Data Bias and Fairness

AI systems are only as objective as the data they're trained on—human-provided data is often highly biased, reflecting societal prejudices.

Data Bias

Models can perpetuate biases if not addressed, e.g., facial recognition systems performing poorly on certain ethnicities due to imbalanced training data.

Mitigation involves diverse datasets, fairness audits, and techniques like reweighting samples.

Slide credit: K. Saenko

Summary

This introduction covers the course structure, including exam details.

Key concepts reviewed:

- **Artificial intelligence:** Broad field encompassing human-like thinking, rational agents, and subfields like NLP and vision.
- **Machine learning:** Subfield focused on learning from data to improve task performance, via supervised, unsupervised, and other paradigms.
- **Deep learning:** Advanced ML using deep neural networks for feature extraction in complex tasks.

Learning fundamentally involves building a model from data.

How to deal with data:

- Observe samples to understand content.
- Feature representation to encode meaningfully.
- Address dataset bias and fairness to ensure equity.

Itates the steps for dealing with data and the Machine Learner Pseudo-Algorithm for a complete workflow.

XKCD: <https://xkcd.com/1838/> (Emphasizing the experimental nature of ML.)

Probabilistic reasoning underpins handling uncertainty throughout.

References

- [Artificial Intelligence](#)
- [Machine Learning](#)
- [Neural Networks](#)
- [Reinforcement Learning](#)
- [Linear Algebra](#) (for ML prerequisites)
- [Probability in AI](#)